

```
from google.colab import drive
drive.mount('/content/drive')
```

↗ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

This dataset is designed for research and analysis of load balancing in distributed systems. It includes key features such as task size, CPU and memory demand, network latency, I/O operations, disk usage, number of connections, and priority level, along with a target variable for classification or optimization. Timestamp data is also provided for temporal analysis. It is suitable for machine learning, simulation studies, and performance optimization research.

Columns:

task_size: Size of the task (numeric).

cpu_demand: CPU demand of the task (numeric).

memory_demand: Memory demand of the task (numeric).

network_latency: Network latency associated with the task (numeric).

io_operations: Number of I/O operations (numeric).

disk_usage: Disk usage for the task (numeric).

num_connections: Number of active connections for the task (numeric).

priority_level: Priority level assigned to the task (numeric).

target: Target label indicating the outcome or category (binary).

timestamp: Timestamp when the task data was recorded.

```
###!pip install bayesian-optimization
```

```
###!pip install keras-tuner
```

```
###!pip uninstall tensorflow
##!pip install tensorflow==2.12.0
```

```
# Import packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from math import floor
from sklearn.metrics import make_scorer, accuracy_score
from bayes_opt import BayesianOptimization
from sklearn.model_selection import StratifiedKFold

import warnings
warnings.filterwarnings('ignore')
pd.set_option("display.max_columns", None)
```

```
# Suppressing Warnings
import warnings
warnings.filterwarnings('ignore')
```

```
# Importing Pandas and NumPy
import pandas as pd, numpy as np
```

```
# Importing all datasets
LoadBalancerSystem = pd.read_csv("/content/dataset.csv")
LoadBalancerSystem.head(4)
```



	task_size	cpu_demand	memory_demand	network_latency	io_operations	disk_usage	num_connections	priority_level	target	timestamp
0	-0.152124	3.750160	-0.981182	0.251507	-0.471993	1.007026	0.313790	3.050953	1	2023-03-16 03:46:22
1	0.724624	-3.978920	2.022732	1.194530	-0.010304	-2.493867	-0.073875	-1.271258	0	2023-09-02 20:15:54
2	4.650228	1.145925	2.641659	-1.899635	1.187132	4.283652	0.572666	1.243801	1	2022-02-19 08:48:52

Next steps:

[Generate code with LoadBalancerSystem](#)[View recommended plots](#)[New interactive sheet](#)

```
from sklearn.model_selection import train_test_split
```

```
train, test = train_test_split(LoadBalancerSystem, test_size=0.2, random_state=1)
```

```
print(train.shape, test.shape)
```

```
(8542, 10) (2136, 10)
```

```
print("*****")
print("The columns in train data :", train.columns)
print("*****")
print("The columns in test data :", test.columns)
print("*****")
```



```
*****
The columns in train data : Index(['task_size', 'cpu_demand', 'memory_demand', 'network_latency',
    'io_operations', 'disk_usage', 'num_connections', 'priority_level',
    'target', 'timestamp'],
    dtype='object')
*****
The columns in test data : Index(['task_size', 'cpu_demand', 'memory_demand', 'network_latency',
    'io_operations', 'disk_usage', 'num_connections', 'priority_level',
```

```
'target', 'timestamp'],  
dtype='object')  
*****
```

```
train.to_csv("train_load_balancer.csv")
```

```
test.to_csv("test_load_balancer.csv")
```

```
# Importing all datasets  
train_dataset = pd.read_csv("/content/train_load_balancer.csv")  
train_dataset.head(4)
```



Unnamed: 0 task_size cpu_demand memory_demand network_latency io_operations disk_usage num_connections priority_level target

0	4311	-2.884349	-0.304593	1.428882	0.641865	-1.265519	-2.042585	-2.151759	2.248828	0
1	6133	0.163591	-0.054587	0.243658	0.631062	-2.349176	-0.106621	0.016042	2.954929	0
2	2321	1.243694	3.099626	0.301481	0.439303	-3.356344	4.851570	-1.382724	2.085178	1
3	1595	-1.675813	0.516609	-1.972764	1.926940	-1.554229	-2.546848	1.107627	1.629831	1

Next steps:

[Generate code with train_dataset](#)[View recommended plots](#)[New interactive sheet](#)

```
# Importing all datasets  
test_dataset = pd.read_csv("/content/test_load_balancer.csv")  
test_dataset.head(4)
```

Unnamed:
0

Unnamed: 0										
	task_size	cpu_demand	memory_demand	network_latency	io_operations	disk_usage	num_connections	priority_level	target	
0	6138	0.982596	2.560753	0.309671	0.484964	-2.768847	4.956329	0.780822	0.050808	1
1	9271	-0.291851	1.488813	1.230143	1.814476	-2.828979	2.481521	-0.649220	0.193506	1
2	4052	0.371047	-0.104333	2.951606	-0.270315	0.201554	1.474998	0.502292	-0.247073	1
3	3068	-0.619218	-0.772362	0.013539	-1.840224	-1.292816	3.058381	2.911729	-1.782623	0

Next steps:

[Generate code with test_dataset](#)[View recommended plots](#)[New interactive sheet](#)

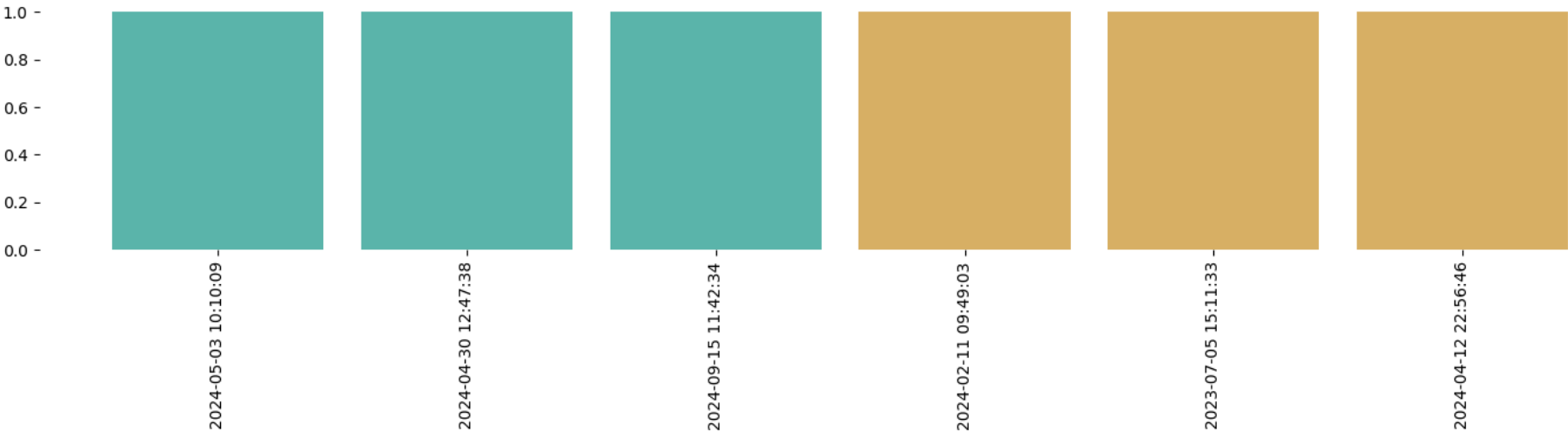
```
####! pip install klib
```

```
import klib
```

```
klib.cat_plot(train_dataset)
```

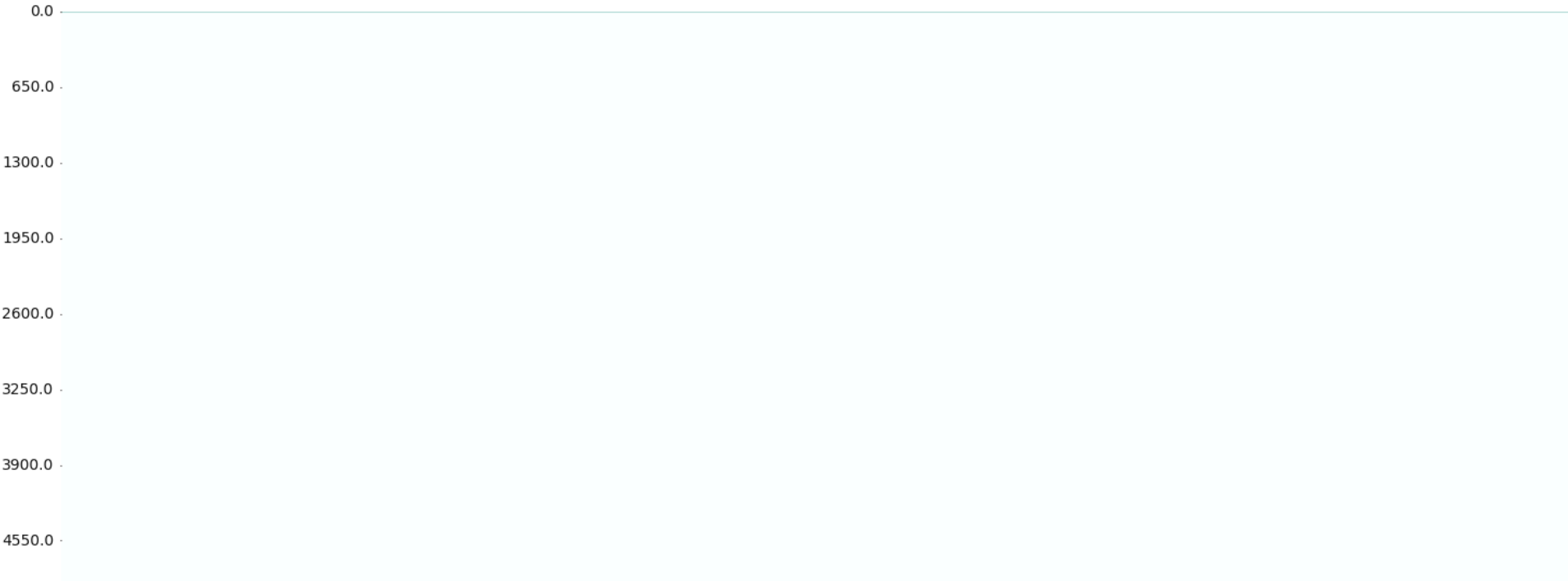
GridSpec(6, 1)

Categorical data plot



Unique values: 8542

Top 3: 3 (0.0%)
Bot 3: 3 (0.0%)





```
klib.corr_mat(train_dataset)
```


 Unnamed:
0

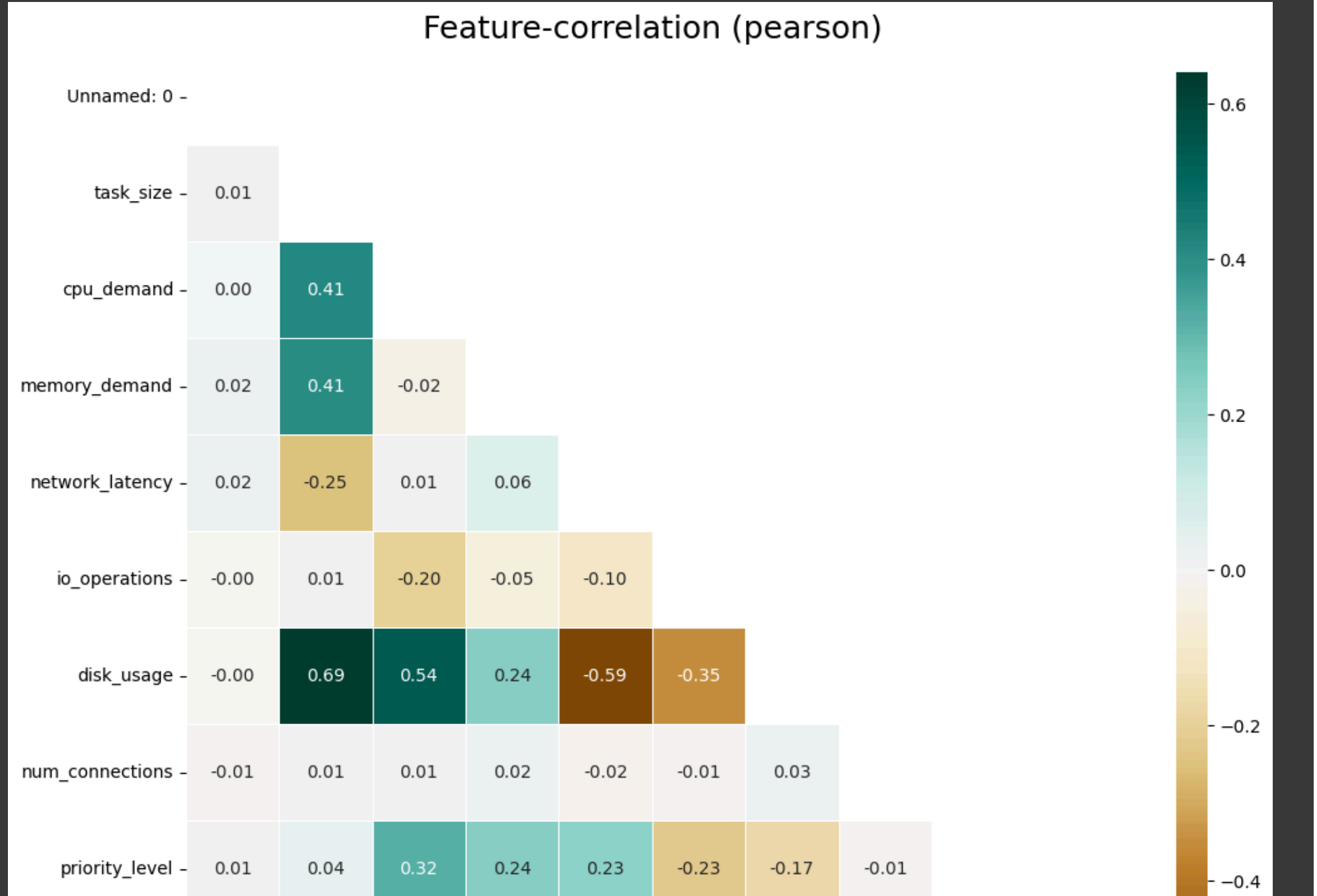
	task_size	cpu_demand	memory_demand	network_latency	io_operations	disk_usage	num_connections	priority_1
Unnamed: 0	1.00	0.01	0.00	0.02	0.02	-0.00	-0.00	-0.01
task_size	0.01	1.00	0.41	0.41	-0.25	0.01	0.69	0.01
cpu_demand	0.00	0.41	1.00	-0.02	0.01	-0.20	0.54	0.01
memory_demand	0.02	0.41	-0.02	1.00	0.06	-0.05	0.24	0.02
network_latency	0.02	-0.25	0.01	0.06	1.00	-0.10	-0.59	-0.02
io_operations	-0.00	0.01	-0.20	-0.05	-0.10	1.00	-0.35	-0.01
disk_usage	-0.00	0.69	0.54	0.24	-0.59	-0.35	1.00	0.03
num_connections	-0.01	0.01	0.01	0.02	-0.02	-0.01	0.03	1.00
priority_level	0.01	0.04	0.32	0.24	0.23	-0.23	-0.17	-0.01
target	0.01	0.30	0.62	-0.01	0.00	0.00	0.27	0.01

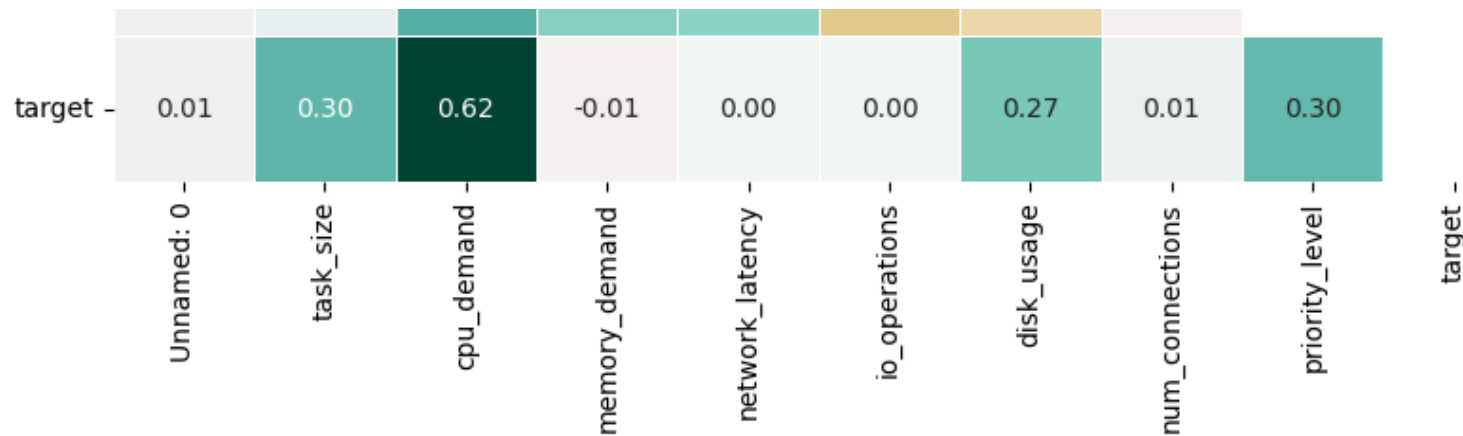
```
plt.figure(figsize=[15,8])
klib.corr_plot(train_dataset)
```



<Axes: title={'center': 'Feature-correlation (pearson)'}>

<Figure size 1500x800 with 0 Axes>





```
klib.missingval_plot(train_dataset)
```

➞ No missing values found in the dataset.

```
train_dataset = klib.data_cleaning(train_dataset)
test_dataset = klib.data_cleaning(test_dataset)
```

➞ Shape of cleaned data: (8542, 11) - Remaining NAs: 0

Dropped rows: 0
of which 0 duplicates. (Rows (first 150 shown): [])

Dropped columns: 0
of which 0 single valued. Columns: []
Dropped missing values: 0
Reduced memory by at least: 0.37 MB (-51.39%)

Shape of cleaned data: (2136, 11) - Remaining NAs: 0

Dropped rows: 0
of which 0 duplicates. (Rows (first 150 shown): [])

Dropped columns: 0
of which 0 single valued. Columns: []
Dropped missing values: 0

Reduced memory by at least: 0.09 MB (-50.0%)

```
train_cleaned = klib.clean_column_names(train_dataset)
test_cleaned = klib.clean_column_names(test_dataset)
```

```
print(train_cleaned.shape, test_cleaned.shape)
```

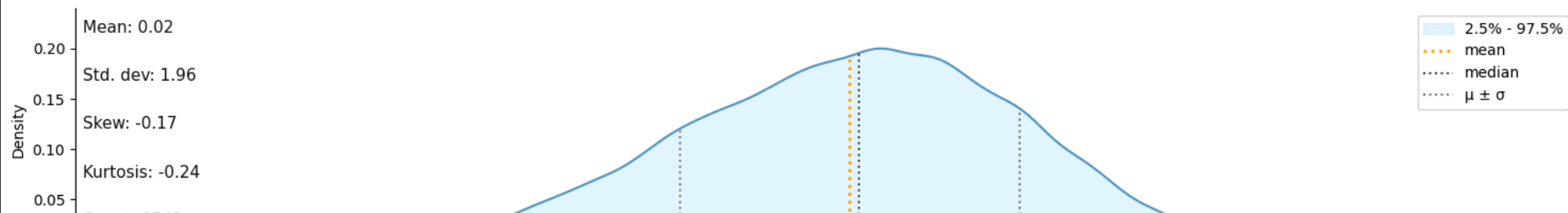
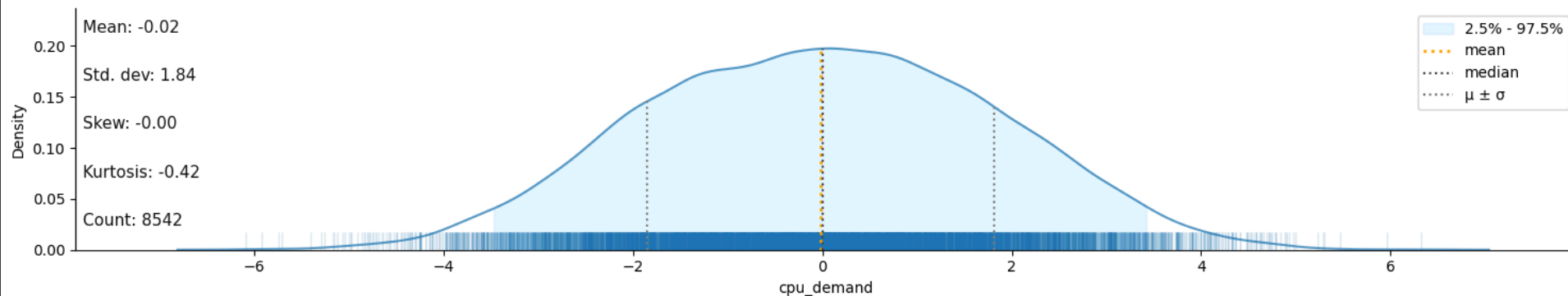
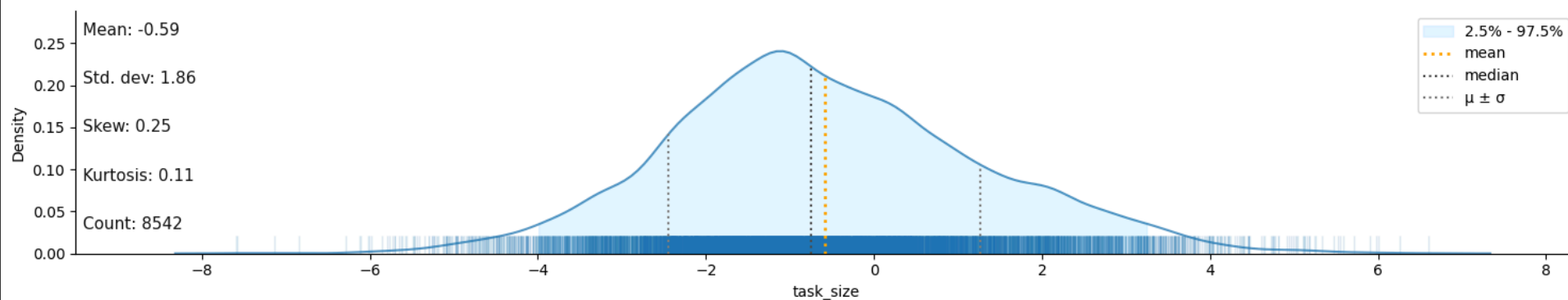
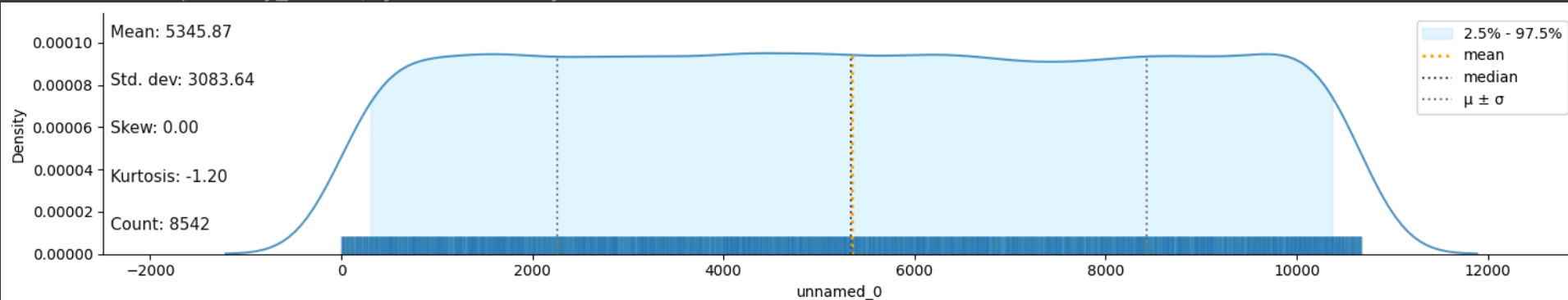
```
➦ (8542, 11) (2136, 11)
```

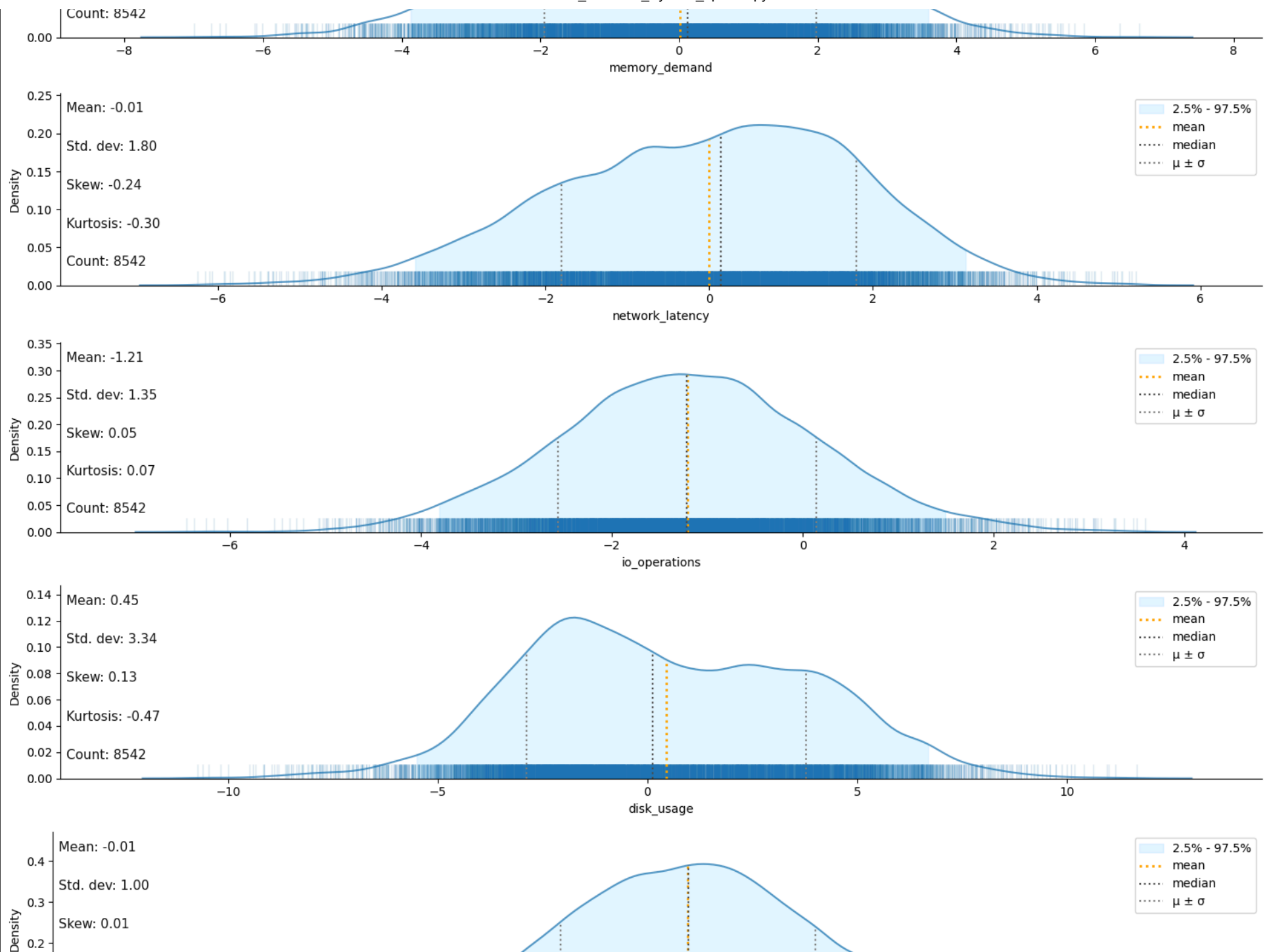
```
train_cleaned = klib.convert_datatypes(train_cleaned)
test_cleaned = klib.convert_datatypes(test_cleaned)
```

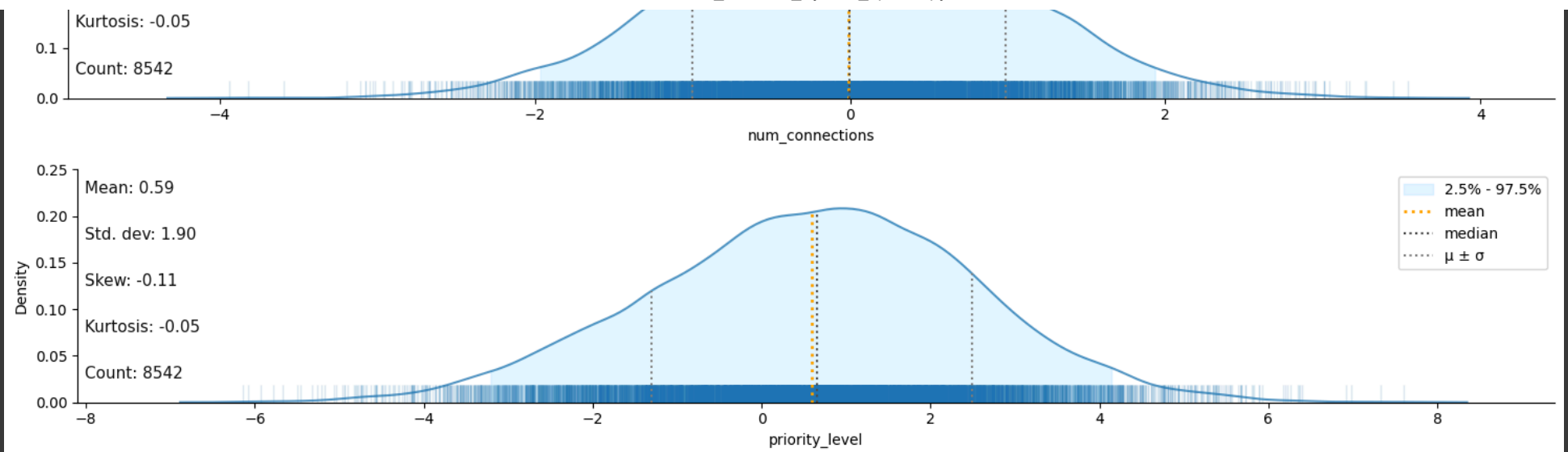
```
klib.dist_plot(train_cleaned)
```



<Axes: xlabel='priority_level', ylabel='Density'>







```
klib.missingval_plot(train_cleaned)
```

➞ No missing values found in the dataset.

```
train_cleaned.dtypes
```

➞

	0
unnamed_0	int16
task_size	float32
cpu_demand	float32
memory_demand	float32
network_latency	float32
io_operations	float32
disk_usage	float32
num_connections	float32
priority_level	float32
target	int8
timestamp	string[python]

dtype: object

```
train_cleaned.columns
```

➞ Index(['unnamed_0', 'task_size', 'cpu_demand', 'memory_demand',
'network_latency', 'io_operations', 'disk_usage', 'num_connections',
'priority_level', 'target', 'timestamp'],
dtype='object')

```
test_cleaned.columns
```

```
Index(['unnamed_0', 'task_size', 'cpu_demand', 'memory_demand',  
      'network_latency', 'io_operations', 'disk_usage', 'num_connections',  
      'priority_level', 'target', 'timestamp'],  
      dtype='object')
```

```
train_cleaned["timestamp"] = pd.to_datetime(train_cleaned["timestamp"])
```

```
train_cleaned["day"] = train_cleaned["timestamp"].dt.day  
train_cleaned["month"] = train_cleaned["timestamp"].dt.month  
train_cleaned["year"] = train_cleaned["timestamp"].dt.year  
train_cleaned["hour"] = train_cleaned["timestamp"].dt.hour
```

```
train_cleaned.drop(columns="timestamp", inplace=True)
```

```
train_cleaned.drop(columns="unnamed_0", inplace=True)
```

```
test_cleaned["timestamp"] = pd.to_datetime(test_cleaned["timestamp"])
```

```
test_cleaned["timestamp"] = pd.to_datetime(test_cleaned["timestamp"])
```

```
test_cleaned["day"] = test_cleaned["timestamp"].dt.day  
test_cleaned["month"] = test_cleaned["timestamp"].dt.month  
test_cleaned["year"] = test_cleaned["timestamp"].dt.year  
test_cleaned["hour"] = test_cleaned["timestamp"].dt.hour
```

```
test_cleaned.drop(columns="timestamp", inplace=True)
```

```
test_cleaned.drop(columns="unnamed_0", inplace=True)
```

```
print(train_cleaned.head(3))
```

```
task_size  cpu_demand  memory_demand  network_latency  io_operations  \  
0  -2.884349   -0.304593      1.428882      0.641865    -1.265519
```

1	0.163591	-0.054587	0.243658	0.631062	-2.349176
2	1.243694	3.099626	0.301481	0.439303	-3.356344

	disk_usage	num_connections	priority_level	target	day	month	year	hour
0	-2.042585	-2.151759	2.248828	0	30	4	2024	12
1	-0.106621	0.016042	2.954928	0	15	9	2024	11
2	4.851570	-1.382725	2.085178	1	20	10	2022	8

```
print(train_cleaned.shape, test_cleaned.shape)
```

```
(8542, 13) (2136, 13)
```

```
X_train = train_cleaned.drop(columns="target")
Y_train = train_cleaned["target"]
```

```
feature_names = X_train.columns
```

```
X_test = test_cleaned.drop(columns="target")
Y_test = test_cleaned["target"]
```

```
print(X_train.shape, Y_train.shape, X_test.shape, Y_test.shape)
```

```
(8542, 12) (8542,) (2136, 12) (2136,)
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Standardize the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
from sklearn.decomposition import PCA
```

```
# Apply PCA to X_train
pca = PCA(n_components=5) # You can choose the number of components
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

```
##! pip install optuna
```

```
import optuna
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier
```

```
# Define the classifiers
classifiers = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC(),
    "k-NN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Gradient Boosting": GradientBoostingClassifier(),
    "XGBoost": XGBClassifier()
}

# Evaluate each classifier using cross-validation
```

```
for name, clf in classifiers.items():
    scores = cross_val_score(clf, X_train_pca, Y_train, cv=5, scoring='accuracy')
    print(f"{name}: Mean Accuracy = {scores.mean():.4f}, Std = {scores.std():.4f}")
```

```
Logistic Regression: Mean Accuracy = 0.7176, Std = 0.0101
Decision Tree: Mean Accuracy = 0.7756, Std = 0.0056
Random Forest: Mean Accuracy = 0.8469, Std = 0.0067
SVM: Mean Accuracy = 0.8468, Std = 0.0095
k-NN: Mean Accuracy = 0.8223, Std = 0.0059
Naive Bayes: Mean Accuracy = 0.7731, Std = 0.0110
Gradient Boosting: Mean Accuracy = 0.8469, Std = 0.0083
XGBoost: Mean Accuracy = 0.8431, Std = 0.0082
```

```
from sklearn.model_selection import train_test_split, cross_val_score
import joblib
import logging
```

```
import xgboost as xgb
```

```
def objective(trial):
    # Define the hyperparameters to tune
    param = {
        'objective': 'binary:logistic',
        'eval_metric': 'logloss',
        'booster': trial.suggest_categorical('booster', ['gbtree', 'gblinear', 'dart']),
        'lambda': trial.suggest_loguniform('lambda', 1e-8, 1.0),
        'alpha': trial.suggest_loguniform('alpha', 1e-8, 1.0),
        'max_depth': trial.suggest_int('max_depth', 1, 10),
        'eta': trial.suggest_loguniform('eta', 1e-8, 1.0),
        'gamma': trial.suggest_loguniform('gamma', 1e-8, 1.0),
        'grow_policy': trial.suggest_categorical('grow_policy', ['depthwise', 'lossguide'])
    }

    if param['booster'] in ['gbtree', 'dart']:
        param['n_estimators'] = trial.suggest_int('n_estimators', 50, 1000)
        param['min_child_weight'] = trial.suggest_int('min_child_weight', 1, 10)
        param['subsample'] = trial.suggest_loguniform('subsample', 0.5, 1.0)
        param['colsample_bytree'] = trial.suggest_loguniform('colsample_bytree', 0.5, 1.0)
```

```
if param['booster'] == 'dart':
    param['sample_type'] = trial.suggest_categorical('sample_type', ['uniform', 'weighted'])
    param['normalize_type'] = trial.suggest_categorical('normalize_type', ['tree', 'forest'])
    param['rate_drop'] = trial.suggest_loguniform('rate_drop', 1e-8, 1.0)
    param['skip_drop'] = trial.suggest_loguniform('skip_drop', 1e-8, 1.0)

# Train the model
dtrain = xgb.DMatrix(X_train_pca, label=Y_train)
dtest = xgb.DMatrix(X_test_pca, label=Y_test)
bst = xgb.train(param, dtrain, evals=[(dtest, 'eval')], verbose_eval=False)

# Make predictions
preds = bst.predict(dtest)
pred_labels = [1 if pred > 0.5 else 0 for pred in preds]

# Calculate accuracy
accuracy = accuracy_score(Y_test, pred_labels)
return accuracy
```

```
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

print("Number of finished trials: ", len(study.trials))
print("Best trial:")
trial = study.best_trial

print("  Value: ", trial.value)
print("  Params: ")
for key, value in trial.params.items():
    print(f"    {key}: {value}")
```

 [I 2025-04-17 12:50:30,171] Trial 3 finished with value: 0.8478464419475655 and parameters: {'booster': 'gbtree', 'lambda': 0.2914295}